

# PostgreSQL & Relational Databases

From set theory and relational logic to advanced PostgreSQL, and building live JavaScript dashboards on top of it.

# What You Will Learn, In Order

Eight modules, building strictly on what came before — no step assumes knowledge you haven't been given yet.

| # | Module                          | You will be able to...                                                |
|---|---------------------------------|-----------------------------------------------------------------------|
| 1 | Set Theory Foundations          | Read and reason about $\cup \cap - \times$ — the math SQL is built on |
| 2 | Relational Database Theory      | Model data with ER diagrams and normalize it correctly                |
| 3 | Getting Started with PostgreSQL | Install Postgres, create databases, and define tables                 |
| 4 | Core SQL Querying               | Write SELECT, WHERE, JOIN, GROUP BY queries confidently               |
| 5 | Intermediate SQL                | Use subqueries, CTEs, window functions, and views                     |
| 6 | Advanced PostgreSQL             | Tune performance, use JSONB, write functions & triggers               |
| 7 | JS/HTML Dashboards on Postgres  | Serve query results to a live browser dashboard                       |
| 8 | Wrap-Up & Reference             | Walk away with cheat sheets and a study plan                          |

# How To Use This Course

- Slides build strictly in order — each concept assumes only what came before it, so resist the urge to skip ahead.
- Every code example is written for PostgreSQL specifically. Small syntax differences exist in MySQL, SQL Server, and Oracle — we will call these out when they matter.
- You do not need to know any programming language to start. We build up from pure logic (set theory) before writing a single line of SQL.
- Speaker notes on each slide contain extra depth — use them if you want the fuller explanation behind the bullets.
- By Module 7 you will need basic familiarity with HTML/JavaScript syntax; we introduce exactly what's needed as we go.

## What You Need

- A computer (Windows, macOS, or Linux)
- About 15-20 hours of study time
- Curiosity, not prior coding experience
- PostgreSQL installed (Module 3 walks through it)

# Why We Start With Math, Not SQL

- SQL is not an arbitrary syntax — it is a practical implementation of the relational model, which itself is built directly on set theory.
- A SQL table IS a mathematical relation: a set of tuples (rows). JOIN, UNION, and INTERSECT are literally set operations wearing SQL syntax.
- Learners who memorize SQL syntax without this foundation can write simple queries but get stuck the moment queries get complex — because they never understood *\*why\** the pieces work.
- Learners who understand the underlying theory can reason about ANY query, even ones they've never seen, because they understand the logic being executed.
- This investment (Modules 1-2) pays for itself many times over in Modules 4-6.

## Relational Model

A way of organizing data as mathematical relations (tables), formalized by Edgar F. Codd in 1970 at IBM. It is the theoretical foundation every SQL database — including PostgreSQL — is built on.

MODULE 1

# 01

## Set Theory Foundations

The pure math underneath every SQL query no database, no syntax, just logic.

- What a set is, and how to write one
- The four core operations:  $\cup$   $\cap$  – complement
- The Cartesian product — the ancestor of every JOIN
- Subsets, and how tables become sets of rows

# What Is a Set?

- A set is simply a well-defined collection of distinct objects, called its elements or members.
- “Well-defined” means: for any object, you can say for certain whether it IS or IS NOT in the set. No ambiguity allowed.
- “Distinct” means: a set never contains duplicates. The set  $\{1, 2, 2, 3\}$  is just the set  $\{1, 2, 3\}$ .
- Order does not matter in a set:  $\{1, 2, 3\}$  and  $\{3, 1, 2\}$  are the exact same set.
- Sets can contain anything: numbers, letters, names, or even rows of a database table — which is exactly where we're headed.

## Set

A collection of distinct, unordered elements. Written with curly braces, e.g.  $S = \{2, 4, 6, 8\}$ . We say  $4 \in S$  (“4 is an element of  $S$ ”) and  $5 \notin S$  (“5 is not an element of  $S$ ”).

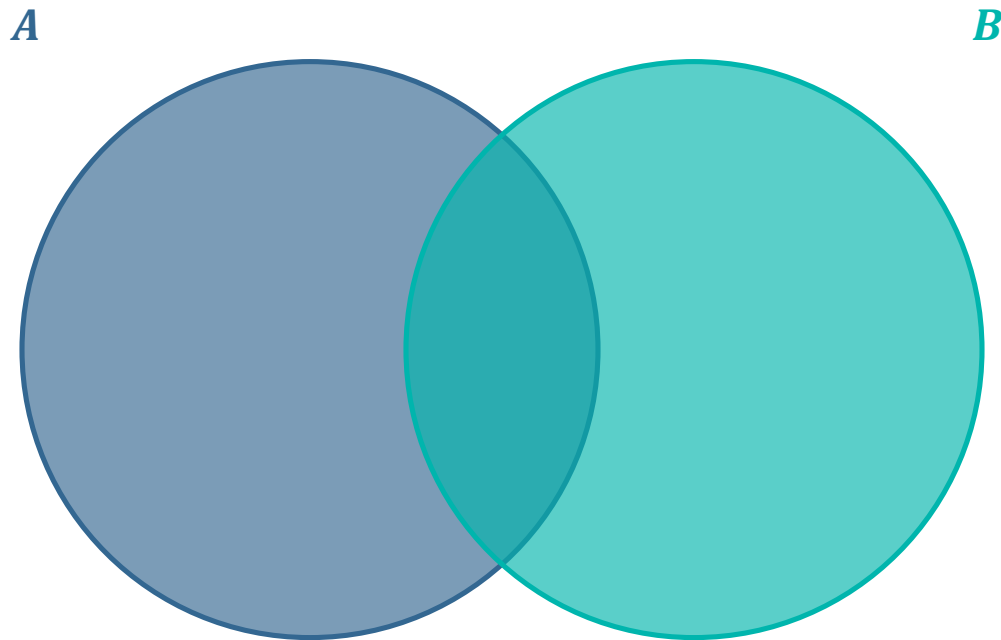
# Writing and Describing Sets

- Roster notation lists every element directly:  $A = \{\text{apple}, \text{banana}, \text{cherry}\}$ .
- Set-builder notation describes a rule instead of listing members:  $A = \{x \mid x \text{ is a fruit in the basket}\}$  (read: “the set of all  $x$  such that  $x$  is a fruit in the basket”).
- The empty set, written  $\emptyset$  or  $\{\}$ , contains no elements at all — it is still a perfectly valid set.
- The cardinality of a set, written  $|A|$ , is simply its number of elements.  $|\{\text{apple}, \text{banana}, \text{cherry}\}| = 3$ .
- Set-builder notation is the direct ancestor of SQL's WHERE clause: `SELECT * FROM fruit WHERE type = 'citrus'` is set-builder notation with SQL syntax.

## THE CONNECTION

```
-- Set-builder:  
{ x | x ∈ Fruits, x.type = 'citrus' }  
  
-- SQL (same idea):  
SELECT *  
FROM fruits  
WHERE type = 'citrus';
```

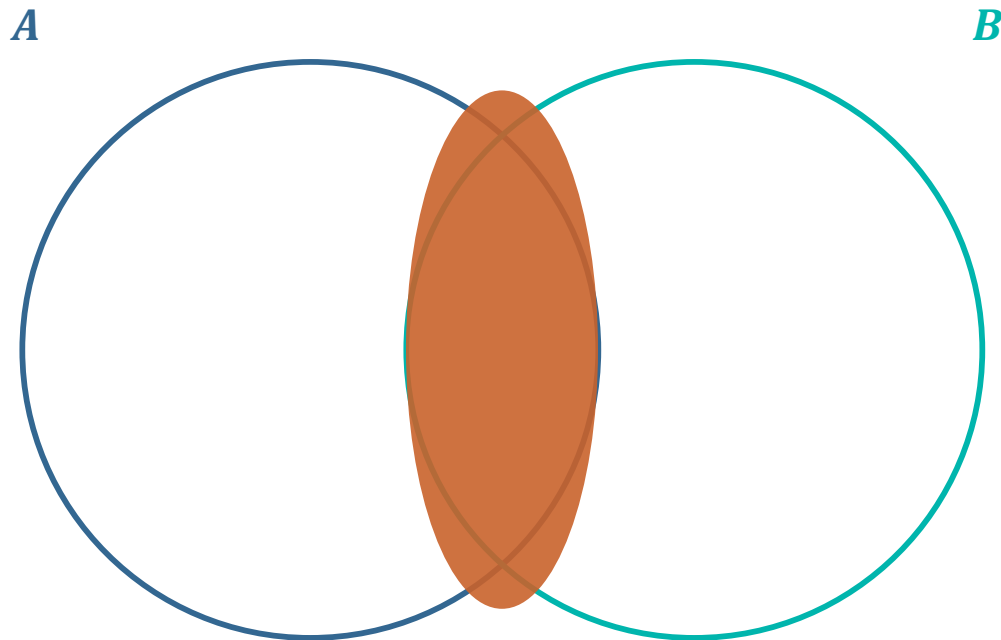
# Union: Combine Everything



## $A \cup B$

- The union of A and B is every element that is in A, in B, or in both — combined into one set.
- Duplicates are never repeated: an element in both sets only appears once in the union.
- Example:  $\{1,2,3\} \cup \{3,4,5\} = \{1,2,3,4,5\}$ .
- SQL equivalent: UNION combines the rows of two result sets into one, removing duplicates.

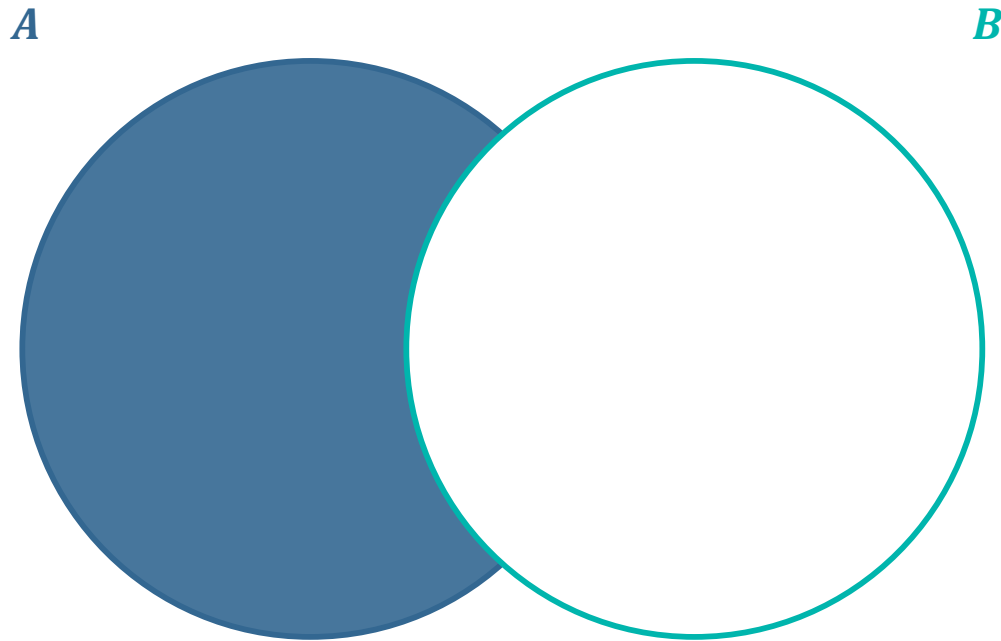
# Intersection: Only What's Shared



## $A \cap B$

- The intersection of A and B is every element that appears in BOTH sets — the overlap only.
- If A and B share nothing, their intersection is the empty set  $\emptyset$ , and they are called disjoint.
- Example:  $\{1,2,3\} \cap \{2,3,4\} = \{2,3\}$ .
- SQL equivalent: INTERSECT returns only rows present in both result sets — and, as we'll see, INNER JOIN is a close cousin of this idea.

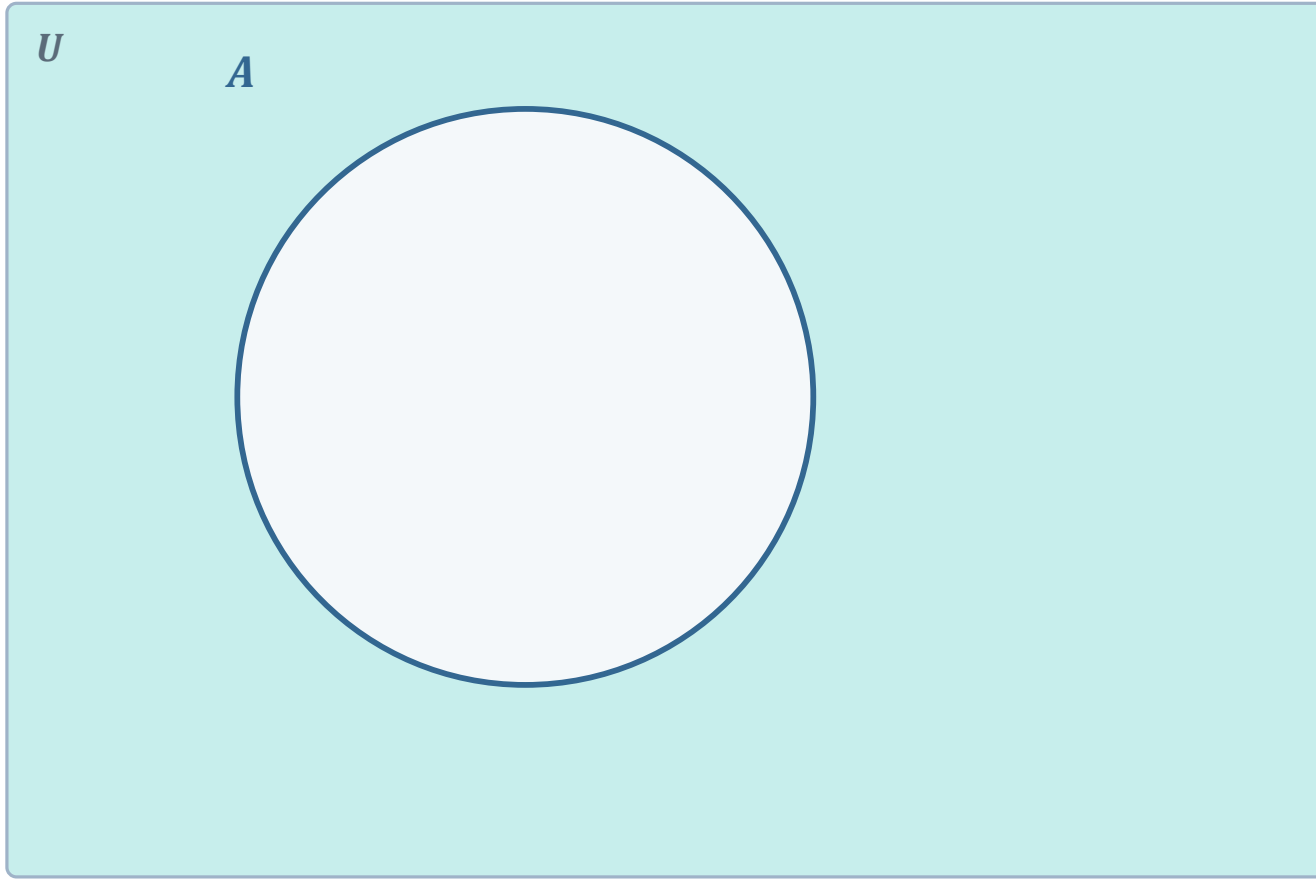
# Difference: What's Only in One



## $A - B$

- The difference  $A - B$  is every element in  $A$  that is NOT also in  $B$  — start with  $A$ , then remove anything shared with  $B$ .
- Difference is not symmetric:  $A - B$  is generally not the same set as  $B - A$ .
- Example:  $\{1,2,3\} - \{2,3,4\} = \{1\}$ , but  $\{2,3,4\} - \{1,2,3\} = \{4\}$ .
- SQL equivalent: EXCEPT returns rows from the first query that don't appear in the second — and this is also the logic behind finding “customers with no orders” using LEFT JOIN ... IS NULL.

# Complement: Everything Else



## $A^c$ (or $U - A$ )

- Every set operation so far compared two sets. Complement needs a third idea: the universe  $U$  — the set of ALL possible elements under discussion.
- The complement of  $A$ , written  $A^c$ , is everything in the universe that is NOT in  $A$ .
- Example: if  $U = \{1..10\}$  and  $A = \{\text{even numbers}\} = \{2,4,6,8,10\}$ , then  $A^c = \{1,3,5,7,9\}$ .
- SQL equivalent: NOT IN, NOT EXISTS, and != all express a complement — “all rows except these.”

# The Cartesian Product: $A \times B$

|    | b1      | b2      | b3      |
|----|---------|---------|---------|
| a1 | (a1,b1) | (a1,b2) | (a1,b3) |
| a2 | (a2,b1) | (a2,b2) | (a2,b3) |

The Cartesian product  $A \times B$  is the set of every possible ordered pair  $(a, b)$  where  $a \in A$  and  $b \in B$ . With  $|A|=2$  and  $|B|=3$ ,  $|A \times B| = 2 \times 3 = 6$  pairs — shown in the grid above. This is the single most important idea for understanding JOINS: a SQL JOIN starts by conceptually building the full Cartesian product of two tables, then filters it down with a condition.

# Subsets and Power Sets

- A is a subset of B (written  $A \subseteq B$ ) if every element of A is also an element of B.
- Every set is a subset of itself, and the empty set  $\emptyset$  is a subset of every set.
- A is a proper subset of B ( $A \subset B$ ) if  $A \subseteq B$  but  $A \neq B$  — B has at least one extra element A doesn't.
- The power set of A, written  $P(A)$ , is the set of ALL possible subsets of A — including  $\emptyset$  and A itself. If  $|A| = n$ , then  $|P(A)| = 2^n$ .
- Subsets map to a simple but powerful SQL idea: a filtered result (WHERE ...) is always a subset of the full table.

**EXAMPLE**

$$A = \{x, y\}$$

Subsets of A:

$\emptyset$

$\{x\}$

$\{y\}$

$\{x, y\}$

$$|P(A)| = 2^2 = 4$$

# A Relation IS a Set of Tuples

- A tuple is simply an ordered list of values, e.g. (“Ada”, “Lovelace”, 1815).
- A relation, in the mathematical sense, is a set of tuples that all share the same structure — the same number and type of positions.
- This is precisely what a database table is: a relation is the table, a tuple is a row, and each position in the tuple is an attribute (column).
- Because a relation is a SET, in the strictest theory it can't contain two identical tuples — real databases relax this slightly (tables CAN have duplicate rows) unless you enforce a key, which we cover next module.
- This single idea — table = relation = set of rows — is why the language is called Structured Query LANGUAGE for RELATIONAL databases.

RELATION = TABLE

| id | name | year |
|----|------|------|
| 1  | Ada  | 1815 |
| 2  | Alan | 1912 |

# Set Theory → SQL: The Cheat Sheet

- You now know every mathematical concept that SQL's core operations are named after or built on.
- Keep this mapping in mind as we move into database theory and then real SQL syntax — you'll see each of these ideas again with a keyword attached.
- Next: we take these abstract sets and give them real-world structure — entities, attributes, keys, and relationships — in Module 2.

## MATH → SQL

- Set → Table (relation)
- Element → Row (tuple)
- Set-builder  $\{x \mid \dots\}$  → WHERE
- $A \cup B$  → UNION
- $A \cap B$  → INTERSECT / INNER JOIN
- $A - B$  → EXCEPT
- $A \times B$  → the basis of JOIN
- Subset → filtered result

MODULE 2

# 02

## Relational Database Theory

How to model real-world information as clean, correct, non-redundant tables.

- Relations, tuples, attributes, and domains
- Primary keys, foreign keys, and candidate keys
- Entity-Relationship (ER) modeling, step by step
- Normalization: 1NF through BCNF

# What Is a Database?

- A database is an organized, persistent collection of related data, stored so it can be efficiently accessed, managed, and updated.
- A Database Management System (DBMS) is the software that stores, retrieves, and protects that data — PostgreSQL is a DBMS.
- A relational database is a database organized according to the relational model: data lives in tables (relations) linked by shared key values, not by physical pointers or nesting.
- Alternatives exist (document stores, key-value stores, graph databases), but the relational model remains dominant because of its mathematical rigor and query flexibility — which is exactly what we built up in Module 1.

## PostgreSQL

A free, open-source, object-relational DBMS first released in 1996, descended from the POSTGRES project at UC Berkeley (1986). Known for strict standards compliance, extensibility, and advanced features like JSONB, full-text search, and custom types.

# Relations, Tuples, Attributes, Domains

- Relation: a table — a named set of tuples that all share the same structure. (“customers” is a relation.)
- Tuple: a row — a single record within the relation. (One specific customer.)
- Attribute: a column — a named property every tuple has a value for. (“email” is an attribute.)
- Domain: the set of legal values an attribute may hold. (The domain of “age” might be “whole numbers from 0 to 130.”)
- A relation's structure — its name plus its attributes and their domains — is called its schema. The actual data inside it is called an instance.

customers (relation)

| id (attr) | name (attr) | age (attr) |
|-----------|-------------|------------|
| 1         | Ada         | 34         |
| 2         | Alan        | 29         |

# Primary Keys

## The Rule

- A primary key (PK) is one attribute (or a combination of attributes) that uniquely identifies every tuple in a relation.
- No two rows may ever share the same primary key value.
- A primary key can never be NULL — every row must be identifiable.
- A relation has exactly one primary key, chosen from among its candidate keys (next slide).

## In Practice

- Natural key: a real-world attribute that's already unique, e.g. an email address or a national ID number.
- Surrogate key: an artificial, meaningless identifier created just to serve as the key — typically an auto-incrementing integer or a UUID.
- Most PostgreSQL schemas default to a surrogate key, e.g. `id SERIAL PRIMARY KEY` or `id UUID PRIMARY KEY`, because natural keys can change or turn out not to be as unique as assumed.

# Foreign Keys

## The Rule

- A foreign key (FK) is an attribute (or set of attributes) in one relation that references the primary key of another (or the same) relation.
- It is how relationships between tables are physically represented in the relational model.
- Every foreign key value must either match an existing primary key value in the referenced table, or be NULL — this rule is called referential integrity.
- A row that violates referential integrity (pointing to a primary key that doesn't exist) is never allowed by the database.

## Worked Example

- `orders.customer_id` is a foreign key referencing `customers.id`.
- Every order **MUST** belong to a customer that actually exists in the customers table.
- This is exactly the subset idea from Module 1: the set of `customer_id` values used in orders must be a subset of the set of `id` values in customers.

# Candidate Keys and Super Keys

- A super key is ANY set of attributes that uniquely identifies a tuple — it may contain more attributes than strictly necessary.
- A candidate key is a MINIMAL super key: remove any single attribute from it, and it stops being unique. Every candidate key is a super key, but not every super key is a candidate key.
- A relation can have several candidate keys. Example: an “employees” table might have both `employee_id` and `social_security_number` as separate candidate keys.
- The primary key is simply whichever candidate key the designer chooses as “the” official identifier. The others remain unique but are called alternate keys.
- In PostgreSQL, alternate keys are enforced with a `UNIQUE` constraint even though they aren't the `PRIMARY KEY`.

## Minimal

A key is minimal when no attribute can be removed from it without losing uniqueness. `{employee_id}` is minimal; `{employee_id, name}` is a super key but NOT minimal, because `employee_id` alone was already enough.

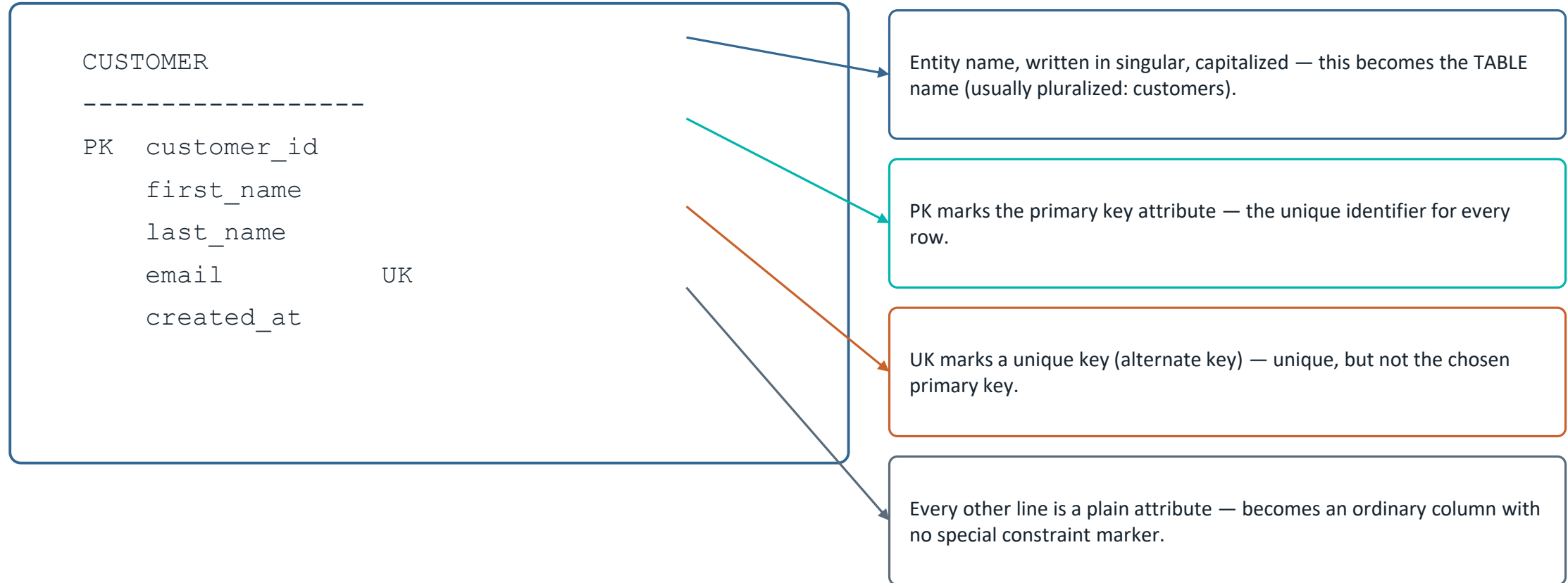
# Entity-Relationship Modeling: The Idea

- Before creating tables, database designers first draw an Entity-Relationship (ER) diagram — a visual map of the data, independent of any specific database syntax.
- An entity is a “thing” worth tracking data about — a customer, a product, an order. Each entity becomes a table.
- An attribute is a property of an entity — a customer's name, a product's price. Each attribute becomes a column.
- A relationship describes how two entities are connected — a customer PLACES an order. Relationships become foreign keys.
- ER modeling is deliberately done on paper/whiteboard BEFORE writing CREATE TABLE — fixing a design mistake in a diagram costs nothing; fixing it after production data exists is expensive.

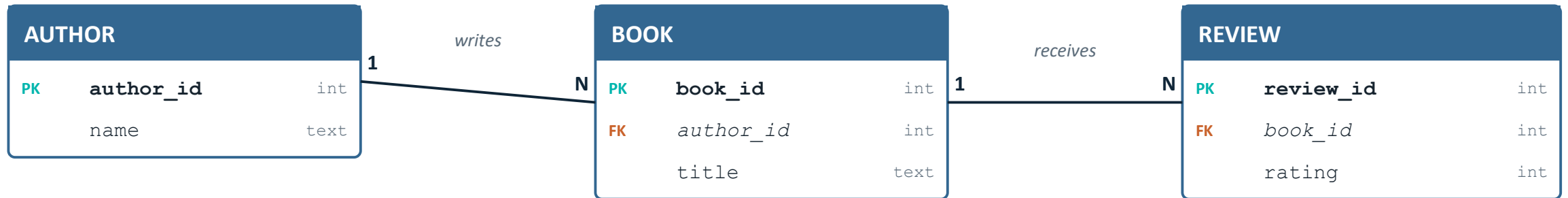
## THE THREE BUILDING BLOCKS

- Entity → becomes a TABLE
- Attribute → becomes a COLUMN
- Relationship → becomes a FOREIGN KEY

# Anatomy/Structure of an Entity



# Relationships and Cardinality



One-to-Many (1:N) is the most common relationship: one author writes MANY books, but each book has (in this simplified model) exactly one author. The “many” side always holds the foreign key — notice `author_id` lives on BOOK, not the other way around.

# The Three Cardinalities

## One-to-One (1:1)

- Each row in A relates to exactly one row in B, and vice versa.
- Example: person  $\leftrightarrow$  passport.
- Implemented with a FK carrying a UNIQUE constraint.
- Rare — often a sign the two tables could be merged.

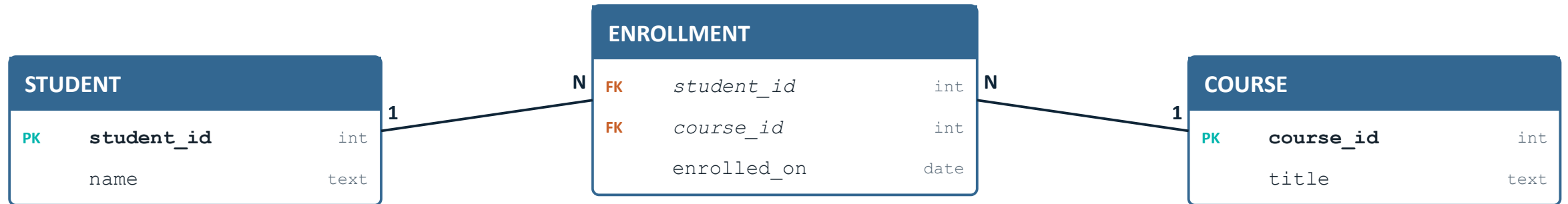
## One-to-Many (1:N)

- One row in A relates to many rows in B; each B row relates to only one A row.
- Example: customer  $\rightarrow$  orders.
- The most common relationship by far.
- Implemented with a plain FK on the “many” table.

## Many-to-Many (M:N)

- Many rows in A relate to many rows in B, and vice versa.
- Example: students  $\leftrightarrow$  courses.
- Cannot be stored with a single FK — needs a junction (bridge) table.
- Junction table holds two FKs, one to each side.

# Modeling Many-to-Many With a Junction Table



ENROLLMENT is a junction table (also called a bridge or associative table). It breaks one M:N relationship into two clean 1:N relationships. Its primary key is usually the COMBINATION of both foreign keys (`student_id`, `course_id`) — a composite key — since one student can only enroll in a given course once.

# Why Normalize? The Three Anomalies

- Normalization is the process of organizing tables to minimize redundancy and eliminate data inconsistency. It's a series of formal rules, applied step by step.
- A poorly designed table — one that stuffs unrelated facts together — suffers from three classic problems, called anomalies.
- Insertion anomaly: you can't add a new fact (e.g., a new course) without also inventing unrelated data (e.g., a fake student) just to have a row to put it in.
- Update anomaly: the same fact is stored in multiple rows, so updating it means finding and changing EVERY copy — miss one, and your data contradicts itself.
- Deletion anomaly: deleting one fact accidentally destroys another unrelated fact that happened to be stored in the same row.

## Redundancy

The same piece of information stored in more than one place. Some redundancy (like a foreign key) is required by design. Uncontrolled redundancy — like repeating a customer's full address on every single order row — is what normalization removes.

# First Normal Form (1NF)

BEFORE — violates 1NF

| order_id | customer | items         |
|----------|----------|---------------|
| 1        | Ada      | laptop, mouse |
| 2        | Alan     | monitor       |



orders

| order_id | customer |
|----------|----------|
| 1        | Ada      |
| 2        | Alan     |

order\_items

| order_id | item    |
|----------|---------|
| 1        | laptop  |
| 1        | mouse   |
| 2        | monitor |

- Rule: every column must hold a single, atomic (indivisible) value — never a list, and never a repeating group of columns.
- The 'items' column above breaks this rule by cramming multiple values into one cell.
- Fix: split the repeating data into its own table, linked back with a foreign key (order\_id).

## Second Normal Form (2NF)

BEFORE — violates 2NF (composite PK: order\_id + item)

| order_id | item   | qty | customer_name |
|----------|--------|-----|---------------|
| 1        | laptop | 1   | Ada           |
| 1        | mouse  | 2   | Ada           |



orders

| order_id | customer_name |
|----------|---------------|
| 1        | Ada           |

order\_items

| order_id | item   | qty |
|----------|--------|-----|
| 1        | laptop | 1   |
| 1        | mouse  | 2   |

- Rule (applies to tables with a composite primary key): every non-key column must depend on the WHOLE key, not just part of it.
- Above, customer\_name depends only on order\_id (part of the key), not on item — that's a partial dependency, and it violates 2NF.
- Fix: move customer\_name to a table keyed by order\_id alone, where it truly belongs.
- 1NF must already be satisfied before 2NF is even meaningful — the normal forms are cumulative.

# Third Normal Form (3NF)

BEFORE — violates 3NF

| order_id | customer_id | customer_email |
|----------|-------------|----------------|
| 1        | 7           | ada@mail.com   |
| 2        | 7           | ada@mail.com   |



orders

| order_id | customer_id |
|----------|-------------|
| 1        | 7           |
| 2        | 7           |

customers

| customer_id | email        |
|-------------|--------------|
| 7           | ada@mail.com |

- Rule: every non-key column must depend ONLY on the primary key — never on another non-key column. This forbidden pattern is called a transitive dependency.
- Above, customer\_email depends on customer\_id, and customer\_id happens to sit in the orders table — so email is transitively (indirectly) dependent on order\_id, not directly.
- Fix: move email to the customers table, where customer\_id is the actual key it depends on.
- Memory aid: “Every non-key attribute must depend on the key, the whole key, and nothing but the key” — a famous summary of 1NF+2NF+3NF together.

# Beyond 3NF, and When to Stop

## BCNF

- Boyce-Codd Normal Form — a stricter version of 3NF.
- Fixes rare edge cases where a table can satisfy 3NF but still has a subtle anomaly, involving overlapping candidate keys.
- In practice, most well-designed 3NF schemas are already in BCNF.

## 4NF & 5NF

- 4NF removes independent multi-valued facts wrongly stored together.
- 5NF handles rare cases of data reconstructable only by joining 3+ tables.
- Mostly academic — rarely a practical concern in everyday schema design.

## Denormalization

- Deliberately reintroducing SOME redundancy for read performance, after normalizing first.
- Common in reporting/analytics tables and dashboards (Module 7!).
- A trade-off, not a mistake — done knowingly, after understanding what's being traded away.

# Relational Theory: Quick Reference

| Concept                      | One-line definition                                                       |
|------------------------------|---------------------------------------------------------------------------|
| Relation / Tuple / Attribute | Table / Row / Column                                                      |
| Primary Key                  | Uniquely identifies each row; never NULL                                  |
| Foreign Key                  | References a primary key in another table; enforces referential integrity |
| Candidate Key                | A minimal super key; a relation may have several                          |
| 1NF                          | Atomic values only — no lists, no repeating groups                        |
| 2NF                          | No partial dependency on part of a composite key                          |
| 3NF                          | No transitive dependency on a non-key column                              |
| Junction Table               | Resolves M:N relationships into two 1:N relationships                     |

MODULE 3

# 03

## Getting Started with PostgreSQL

From zero to a running database with your first table, on your own machine.

- Installing PostgreSQL on any OS
- Touring psql and pgAdmin
- Data types, CREATE TABLE, and constraints
- The sample schema used for the rest of the course

# Installing PostgreSQL

## Windows

- Download the installer from [postgresql.org/download](https://www.postgresql.org/download).
- Run it; accept defaults for install directory and port (5432).
- Set a password for the postgres superuser — write it down.
- Installer bundles pgAdmin automatically.

## macOS

- Easiest: `brew install postgresql@16` (via Homebrew).
- Or use the Postgres.app graphical installer.
- Start the service: `brew services start postgresql@16`.
- pgAdmin installed separately if wanted.

## Linux

- Debian/Ubuntu: `sudo apt install postgresql postgresql-contrib`.
- The service starts automatically after install.
- Default admin OS user is 'postgres': `sudo -u postgres psql`.
- Fedora/RHEL: `sudo dnf install postgresql-server`.

# Touring psql

**psql** is PostgreSQL's official interactive command-line client the fastest way to work directly with your database.

```
$ psql -U postgres -h localhost
Password: *****
postgres=# \l                      -- list all databases
postgres=# \c mydb                 -- connect to database "mydb"
mydb=# \dt                        -- list tables in current database
mydb=# \d customers               -- describe the "customers" table's columns
mydb=# \du                        -- list roles / users
mydb=# \q                         -- quit psql

mydb=# SELECT version();          -- ordinary SQL always ends in a semicolon
```

*Backslash commands (\l, \dt, \d, \q ...) are psql-specific shortcuts, NOT standard SQL — SQL statements always end with a semicolon.*

# Touring pgAdmin

- pgAdmin is the official graphical administration tool for PostgreSQL — a browser-based UI that runs locally.
- The left-hand Object Browser tree lets you navigate Servers → Databases → Schemas → Tables visually, without memorizing commands.
- The Query Tool (a SQL editor with syntax highlighting and autocomplete) is where you'll write and run most of the SQL in this course.
- Right-click any table → View/Edit Data to browse and edit rows in a spreadsheet-like grid — useful for quickly checking results.
- Either psql or pgAdmin works fine for this entire course — use whichever you're more comfortable with; the SQL itself is identical.

## PGADMIN VS PSQL

- pgAdmin: visual, beginner-friendly, point-and-click
- psql: fast, scriptable, works over SSH
- Both talk to the same Postgres server
- Professionals commonly use both

# Creating Your First Database

```
-- From the command line, before connecting to a specific database:  
CREATE DATABASE course_db;  
  
-- Then connect to it:  
\c course_db  
  
-- Or, from the shell directly:  
$ createdb course_db  
$ psql -d course_db
```

# PostgreSQL Data Types, at a Glance

Every column must be given a data type when the table is created — this is the formal “domain” concept from Module 2, made concrete.

| Category        | Common types                         | Example use                     |
|-----------------|--------------------------------------|---------------------------------|
| Whole numbers   | SMALLINT, INTEGER, BIGINT, SERIAL    | Ages, counts, IDs               |
| Decimal numbers | NUMERIC(p,s), REAL, DOUBLE PRECISION | Prices, measurements            |
| Text            | VARCHAR(n), TEXT, CHAR(n)            | Names, descriptions             |
| Boolean         | BOOLEAN                              | Flags: is_active, is_paid       |
| Date & time     | DATE, TIME, TIMESTAMP, TIMESTAMPTZ   | Order dates, timestamps         |
| Structured      | JSON, JSONB, ARRAY                   | Flexible/nested data (Module 6) |
| Identifiers     | UUID                                 | Globally unique surrogate keys  |

## CREATE TABLE, Piece by Piece

```
CREATE TABLE customers (  
    id          SERIAL PRIMARY KEY,  
    first_name  VARCHAR(50)  NOT NULL,  
    last_name   VARCHAR(50)  NOT NULL,  
    email       VARCHAR(255) UNIQUE NOT NULL,  
    created_at  TIMESTAMPTZ  DEFAULT now()  
);
```

*SERIAL auto-generates an incrementing integer — Postgres's classic surrogate-key shorthand (modern style: GENERATED ALWAYS AS IDENTITY).*

# Column Constraints

## Value Rules

- NOT NULL — this column can never be left empty.
- UNIQUE — no two rows may share the same value here.
- CHECK (condition) — a custom rule, e.g. CHECK (price > 0).
- DEFAULT value — used automatically if no value is given on insert.

## Key Constraints

- PRIMARY KEY — combines NOT NULL + UNIQUE, and marks the row's identifier.
- FOREIGN KEY ... REFERENCES ... — enforces that a value must exist in another table's primary key (or be NULL).
- Constraints can be added inline (next to a column) or as a separate table-level clause — both are shown on the next slide.

# Primary and Foreign Keys in Practice

```
CREATE TABLE orders (  
    id            SERIAL PRIMARY KEY,  
    customer_id   INTEGER NOT NULL,  
    total_cents   INTEGER NOT NULL CHECK (total_cents >= 0),  
    ordered_at    TIMESTAMPTZ DEFAULT now(),  
  
    CONSTRAINT fk_customer  
        FOREIGN KEY (customer_id)  
        REFERENCES customers(id)  
        ON DELETE CASCADE  
  
);
```

***ON DELETE CASCADE means: if a customer row is deleted, all of their orders are automatically deleted too. Other options: ON DELETE RESTRICT (block the delete) or SET NULL.***

# The INSERT Statement

```
INSERT INTO customers (first_name, last_name, email)
VALUES ('Ada', 'Lovelace', 'ada@example.com');
```

```
-- Insert several rows at once:
```

```
INSERT INTO customers (first_name, last_name, email)
VALUES
    ('Alan', 'Turing', 'alan@example.com'),
    ('Grace', 'Hopper', 'grace@example.com');
```

```
-- Return the row(s) just created:
```

```
INSERT INTO customers (first_name, last_name, email)
VALUES ('Edsger', 'Dijkstra', 'edsger@example.com')
RETURNING id, created_at;
```

*id and created\_at are never listed in VALUES here — SERIAL and DEFAULT fill them in automatically.*

# The Sample Database Used For The Rest of This Course



This four-table e-commerce schema (customers → orders → order\_items → products) is used in every SQL example from Module 4 onward. order\_items is the familiar junction-style table connecting orders to products — exactly the pattern from slide 2.10.

MODULE 4

# 04

## Core SQL Querying

**SELECT, WHERE, JOIN, and GROUP BY** the queries you'll write every single day.

- Filtering, sorting, and limiting results
- Aggregate functions and GROUP BY / HAVING
- Every JOIN type, explained with the set diagrams from Module 1
- Multi-table queries on the sample schema

# The SELECT Statement

Every SQL query that reads data starts with SELECT — this is the set-builder notation from Module 1, spelled out in keywords.

```
SELECT *  
FROM customers;  
  
-- '*' means "all columns" — fine for exploring, avoid in real application code  
  
SELECT first_name, last_name, email  
FROM customers;
```

# Column Aliases and Expressions

```
SELECT
    first_name || ' ' || last_name AS full_name,
    email,
    total_cents / 100.0           AS total_dollars
FROM customers
JOIN orders ON orders.customer_id = customers.id;
```

*AS* renames a column (or expression result) in the output. `||` concatenates text in PostgreSQL. Column expressions can do arithmetic, string building, or type conversion directly in

# WHERE and Comparison Operators

- WHERE filters rows — only tuples satisfying the condition are kept. This IS the set-builder predicate from Module 1, executed literally.
- Standard comparisons: = (equals), != or <> (not equal), <, >, <=, >=.
- WHERE is evaluated once per row, and only TRUE rows survive; FALSE and UNKNOWN (from NULL comparisons) are both discarded.
- Multiple conditions combine with AND / OR / NOT, covered next.

## EXAMPLE

```
SELECT *  
FROM products  
WHERE price_cents > 5000;
```

# Combining Conditions: AND, OR, NOT

- AND requires both conditions to be true — narrows the result set (this is set intersection thinking).
- OR requires at least one condition to be true — widens the result set (set union thinking).
- NOT reverses a condition entirely (set complement thinking).
- Parentheses control evaluation order, exactly like in math — AND binds tighter than OR, so always parenthesize mixed AND/OR conditions to avoid surprises.

## WATCH THE PARENTHESES

-- Ambiguous intent:

```
WHERE city = 'NYC'
```

```
    OR city = 'LA'
```

```
    AND vip = true
```

-- Explicit intent:

```
WHERE (city = 'NYC' OR city = 'LA')
```

```
    AND vip = true
```

# BETWEEN, IN, and LIKE

## BETWEEN

- WHERE price BETWEEN 10 AND 50
- Inclusive on both ends.
- Shorthand for price  $\geq 10$  AND price  $\leq 50$ .

## IN

- WHERE status IN ('shipped','paid')
- Shorthand for chained OR = comparisons.
- Set membership, straight from Module 1.

## LIKE

- WHERE name LIKE 'A%'
- % matches any sequence; \_ matches one character.
- ILIKE is PostgreSQL's case-insensitive version.

# Handling NULL Correctly

- NULL means “unknown” or “absent” — it is NOT the same as zero, an empty string, or false.
- Any comparison against NULL using =, !=, <, > returns UNKNOWN, not TRUE or FALSE — so WHERE email = NULL NEVER matches any row, even rows where email really is NULL.
- Use IS NULL / IS NOT NULL to test for NULL correctly.
- COALESCE(value, fallback) returns the first non-NULL argument — the standard way to substitute a default when a value is missing.
- This three-valued logic (TRUE / FALSE / UNKNOWN) is one of the most common sources of subtle SQL bugs — internalize it now.

## EXAMPLE

```
SELECT name,  
       COALESCE(phone, 'no phone on file')  
AS phone  
FROM customers  
WHERE phone IS NOT NULL;
```

# Sorting Results

```
SELECT name, price_cents
FROM products
ORDER BY price_cents DESC;      -- highest first
```

```
SELECT name, price_cents
FROM products
ORDER BY category ASC, price_cents DESC;  -- multi-column sort
```

*ASC (ascending, default) or DESC (descending). Recall Module 1: a set has no inherent order — without ORDER BY, a SQL result's row order is NOT guaranteed, even if it often looks consistent.*

Consistent Sorting & Limiting

# Paging Through Results

```
SELECT name, price_cents
FROM products
ORDER BY price_cents DESC
LIMIT 10;                -- top 10 most expensive products
```

```
SELECT name, price_cents
FROM products
ORDER BY price_cents DESC
LIMIT 10 OFFSET 20;      -- skip first 20, then take the next 10 ("page 3")
```

## DISTINCT: Removing Duplicates

- DISTINCT removes duplicate rows from the result — the direct SQL implementation of a mathematical set's “no duplicates” rule from Module 1.
- `SELECT DISTINCT city FROM customers;` returns each unique city once, no matter how many customers live there.
- DISTINCT considers the ENTIRE row (or all selected columns together) when deciding what counts as a duplicate.
- DISTINCT ON (column) is a PostgreSQL-specific extension that returns the first row (by ORDER BY) for each distinct value of that column — very handy, not standard SQL.

### EXAMPLE

```
SELECT DISTINCT city  
FROM customers  
ORDER BY city;
```

# Summarizing Data: Part 1

## COUNT

- `COUNT(*)` — counts all rows, including NULLs.
- `COUNT(column)` — counts only rows where that column is NOT NULL.
- `COUNT(DISTINCT column)` — counts unique non-null values.

## SUM & AVG

- `SUM(column)` — adds up all non-null numeric values.
- `AVG(column)` — the mean of all non-null numeric values.
- Both ignore NULLs automatically — they don't count as zero.

# Summarizing Data: Part 2

## MIN & MAX

- MIN(column) — the smallest value present.
- MAX(column) — the largest value present.
- Work on numbers, dates, and even text (alphabetical order).

## Without GROUP BY

- An aggregate function with no GROUP BY collapses the ENTIRE table into a single summary row.
- SELECT COUNT(\*), AVG(price\_cents) FROM products; returns exactly one row — totals across everything.

# Grouping Rows Before Aggregating

```
SELECT customer_id, COUNT(*) AS order_count, SUM(total_cents) AS total_spent
FROM orders
GROUP BY customer_id;
```

```
-- Multiple grouping columns:
SELECT category, status, COUNT(*)
FROM products p JOIN orders o ON ...
GROUP BY category, status;
```

*GROUP BY splits the table into buckets sharing the same value(s), then runs the aggregate function separately WITHIN each bucket. Every non-aggregated column in SELECT must appear in GROUP BY.*

## Filtering Groups With HAVING

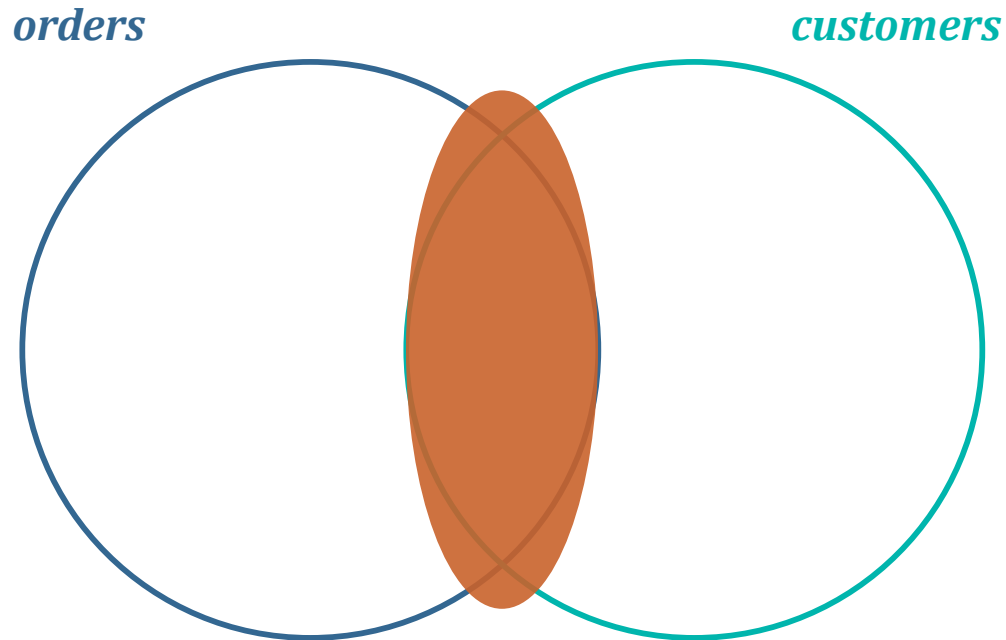
```
SELECT customer_id, COUNT(*) AS order_count
FROM orders
GROUP BY customer_id
HAVING COUNT(*) > 5;      -- only customers with more than 5 orders
```

*WHERE filters individual rows BEFORE grouping happens. HAVING filters entire groups AFTER aggregation — you cannot write WHERE COUNT(\*) > 5, because WHERE runs before COUNT(\*) aggregation*

# The Logical Order of Execution

- 1 FROM / JOIN**  
Identify the source table(s) and combine them if joined.
- 2 WHERE**  
Filter individual rows based on a condition. Aggregates don't exist yet here.
- 3 GROUP BY**  
Bucket the remaining rows into groups.
- 4 HAVING**  
Filter entire groups, now that aggregates like COUNT() can be evaluated.
- 5 SELECT**  
Compute the final column list / expressions, including aggregates and aliases.
- 6 ORDER BY / LIMIT**  
Sort the final result and cut it down to the requested size.

# INNER JOIN = Intersection, Revisited



## **orders ⋈ customers**

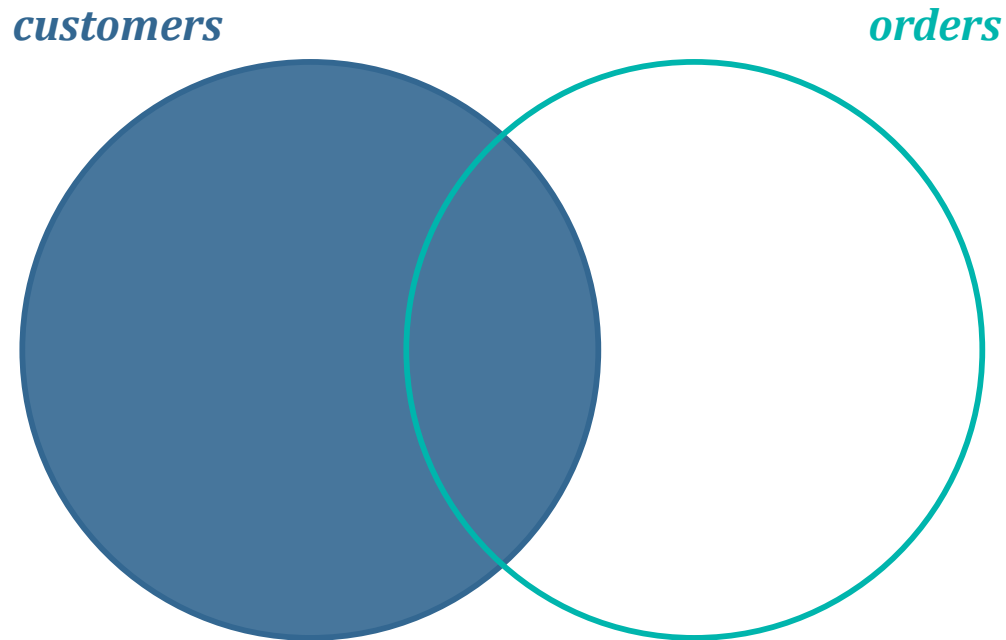
- INNER JOIN returns only rows that have a match in BOTH tables — visually identical to the intersection diagram from Module 1.
- It's the default JOIN type: writing just JOIN in PostgreSQL means INNER JOIN.
- Rows from either table with no matching partner are silently excluded from the result.

## INNER JOIN in Practice

```
SELECT customers.first_name, orders.id, orders.total_cents
FROM orders
INNER JOIN customers
    ON orders.customer_id = customers.id;

-- 'INNER' is optional — identical result:
SELECT customers.first_name, orders.id
FROM orders
JOIN customers ON orders.customer_id = customers.id;
```

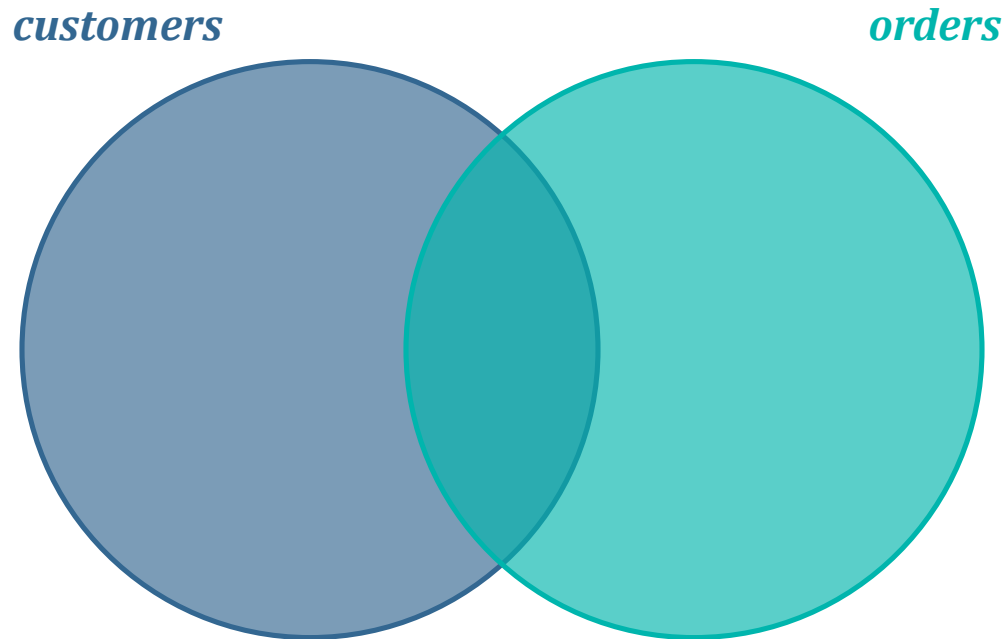
# LEFT JOIN: Keep Everything on the Left



## **customers ⋈ orders**

- LEFT JOIN (or LEFT OUTER JOIN) keeps EVERY row from the left table, whether or not it has a match on the right.
- When there's no match, the right table's columns come back as NULL for that row.
- Classic use case: “find customers with NO orders” — LEFT JOIN then WHERE orders.id IS NULL.
- RIGHT JOIN is the mirror image — keeps everything on the right instead.

# FULL OUTER JOIN and CROSS JOIN



## **customers ⋈ orders**

- FULL OUTER JOIN keeps every row from BOTH tables — matched where possible, NULL-padded where not. Visually, this is the union from Module 1.
- CROSS JOIN (no ON clause) returns the full, unfiltered Cartesian product of two tables — every row of A paired with every row of B.
- CROSS JOIN is rarely used directly, but understanding it makes every other JOIN make sense: they're all 'CROSS JOIN, then filtered.'
- If table A has 100 rows and table B has 50, CROSS JOIN A, B returns 5,000 rows.

# Self Joins: A Table Joined to Itself



`SELECT e.name AS employee, m.name AS manager FROM employees e JOIN employees m ON e.manager_id = m.id;` — the SAME physical table is given two different aliases (e and m) so SQL can treat it as two separate sets and join them against each other. Used constantly for hierarchies (org charts, category trees, comment threads).

# Joining Three Tables

Real queries routinely join more than two tables — chain JOIN clauses one after another.

```
SELECT
    c.first_name,
    o.id           AS order_id,
    p.name         AS product,
    oi.qty
FROM customers c
JOIN orders      o  ON o.customer_id = c.id
JOIN order_items oi ON oi.order_id    = o.id
JOIN products    p  ON p.id           = oi.product_id
WHERE c.email = 'ada@example.com'
ORDER BY o.ordered_at DESC;
```

MODULE 5

# 05

## Intermediate SQL

**Subqueries, CTEs, window functions, and views** the tools that separate confident SQL from beginner SQL.

- Subqueries: scalar, IN/EXISTS, correlated
- CTEs and recursive CTEs
- Window functions: RANK, LAG/LEAD, running totals
- Views, transactions, and indexing basics

# Scalar Subqueries

```
SELECT name, price_cents
FROM products
WHERE price_cents > (
    SELECT AVG(price_cents) FROM products
);      -- products priced above the overall average
```

# IN and EXISTS Subqueries

## IN

- `WHERE customer_id IN (SELECT id FROM customers WHERE vip = true)`
- Checks if a value is a member of the set the subquery returns — pure Module 1 set membership.
- Subquery may return many rows, but only one column.

## EXISTS

- `WHERE EXISTS (SELECT 1 FROM orders WHERE orders.customer_id = customers.id)`
- Checks only whether the subquery returns ANY row at all — doesn't care about the actual values.
- Often faster than IN on large sets because the database can stop at the first match.

# Correlated Subqueries

```
SELECT c.first_name,  
       (SELECT COUNT(*) FROM orders o  
        WHERE o.customer_id = c.id) AS order_count  
FROM customers c;
```

A correlated subquery references a column from the OUTER query (*c.id*) — so it can't run on its own; it re-executes once for every row of the outer query. Powerful, but potentially slower than joins (a JOIN + GROUP BY is often faster for this exact case).

## WITH: Naming a Subquery

```
WITH big_spenders AS (  
    SELECT customer_id, SUM(total_cents) AS total  
    FROM orders  
    GROUP BY customer_id  
    HAVING SUM(total_cents) > 100000  
)  
SELECT c.first_name, bs.total  
FROM big_spenders bs  
JOIN customers c ON c.id = bs.customer_id  
ORDER BY bs.total DESC;
```

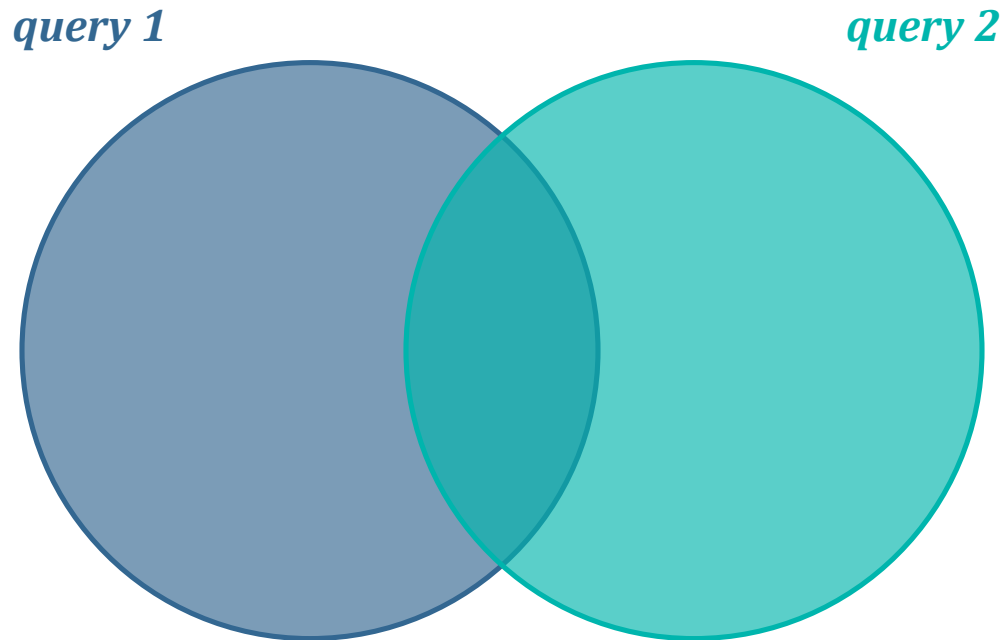
A Common Table Expression (CTE) gives a temporary, named result set that the rest of the query can reference — like a variable holding a table. Improves readability enormously over nested subqueries.

# Recursive CTEs: Walking a Hierarchy

```
WITH RECURSIVE org_chart AS (  
    SELECT id, name, manager_id, 1 AS depth  
    FROM employees WHERE manager_id IS NULL          -- anchor: the CEO  
  
    UNION ALL  
  
    SELECT e.id, e.name, e.manager_id, oc.depth + 1  
    FROM employees e  
    JOIN org_chart oc ON e.manager_id = oc.id        -- recursive step  
)  
SELECT * FROM org_chart ORDER BY depth;
```

A recursive CTE has an anchor query (the starting point) UNION ALL'd with a recursive query that repeatedly joins back to the CTE's own results, until no new rows are produced. Perfect for trees and hierarchies.

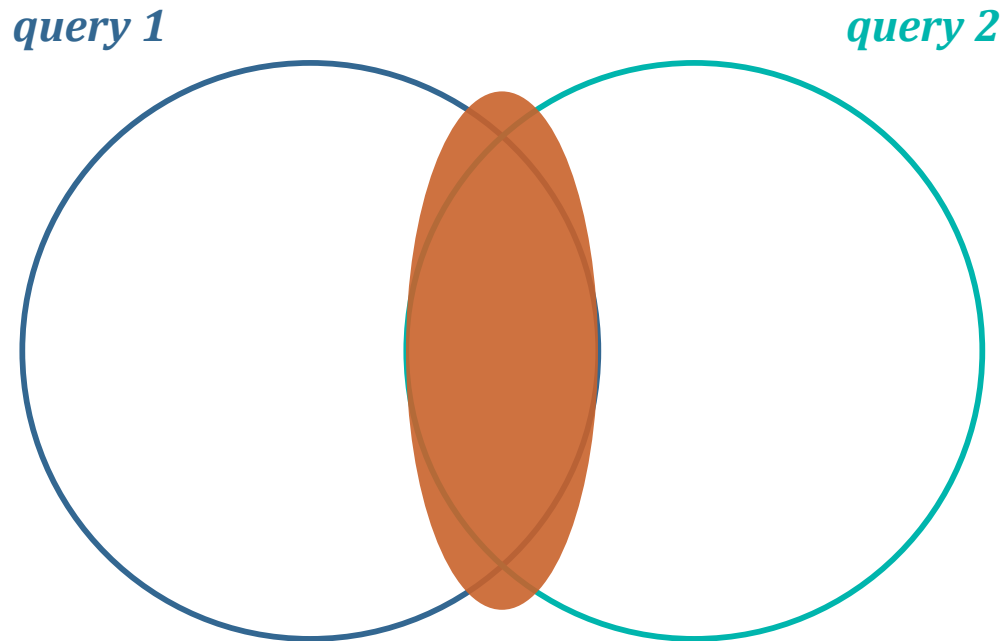
# UNION and UNION ALL



## UNION [ALL]

- UNION combines the rows of two queries into one result AND removes duplicate rows — the exact union operation from Module 1.
- UNION ALL combines rows WITHOUT removing duplicates — faster, since no duplicate-checking work is needed.
- Both queries must return the same number of columns, with compatible types, in the same order.
- Rule of thumb: use UNION ALL unless you specifically need duplicates removed.

# INTERSECT and EXCEPT



## INTERSECT / EXCEPT

- INTERSECT returns only rows present in BOTH queries' results — set intersection, Module 1 again.
- EXCEPT returns rows from the FIRST query that do NOT appear in the second — set difference.
- Both remove duplicates from the final result by default, like UNION.
- Same column-count and type-compatibility rules as UNION apply.

# Views: Saved, Reusable Queries

## What They Are

- `CREATE VIEW customer_totals AS SELECT ...` stores a query under a name.
- Querying the view (`SELECT * FROM customer_totals`) re-runs the underlying query fresh, every time.
- A view stores no data of its own — it's a saved `SELECT` statement, nothing more.

## Why Use Them

- Hide complex `JOIN`/aggregation logic behind a simple, table-like name.
- Restrict which columns/rows a user can see, for basic access control.
- Keep application code decoupled from schema details underneath.

# Views vs. Materialized Views

## View

- Stores the QUERY, not the data.
- Always reflects current data.
- Recomputes on every single SELECT.
- Best for: light queries, always-fresh needs.

## Materialized View

- Stores the RESULT physically on disk, like a cached table.
- Data can go stale until refreshed.
- REFRESH MATERIALIZED VIEW name; recomputes it on demand.
- Best for: expensive aggregations, dashboards (Module 7!).

# Window Functions: The Big Idea

- GROUP BY collapses many rows into one summary row per group. Window functions do something different: they compute an aggregate-like value PER ROW, while keeping every original row intact.
- The syntax pattern is: function(...) OVER (...) — that OVER clause defines the “window” of rows the function can see for each row.
- This solves exactly the problem correlated subqueries were slow at: 'this row's value, plus some context from related rows' — without losing row-level detail.
- Window functions run logically AFTER WHERE/GROUP BY/HAVING but BEFORE ORDER BY/LIMIT — slotting into the execution order from Module 4.

## OVER()

The clause that turns an ordinary aggregate (or a ranking function) into a window function. An empty OVER() considers the entire result set as one window; OVER (PARTITION BY ...) splits it into sub-windows.

# ROW\_NUMBER, RANK, DENSE\_RANK



| name   | score | rank |
|--------|-------|------|
| Ada    | 95    | 1    |
| Alan   | 90    | 2    |
| Grace  | 90    | 2    |
| Edsger | 88    | 4    |

- `ROW_NUMBER()` — always gives a unique, sequential number: 1,2,3,4, even for tied values.
- `RANK()` — ties share the same rank, and a gap follows: 1,2,2,4 (shown here).
- `DENSE_RANK()` — ties share the same rank, but NO gap follows: 1,2,2,3.
- All three use `ORDER BY` inside `OVER()` to define what 'first' means: `RANK() OVER (ORDER BY score DESC)`.

## PARTITION BY: Separate Windows per Group



| dept  | name   | salary_rank |
|-------|--------|-------------|
| Eng   | Ada    | 1           |
| Eng   | Alan   | 2           |
| Sales | Grace  | 1           |
| Sales | Edsger | 2           |

- PARTITION BY splits rows into independent groups, and the window function restarts fresh WITHIN each partition — notice rank resets to 1 for each dept.
- RANK() OVER (PARTITION BY dept ORDER BY salary DESC)
- This is conceptually identical to GROUP BY's bucketing, but rows are NOT collapsed — every original row is preserved.
- Extremely common pattern: 'top N per category', 'this row vs. its group average', etc.

## LAG and LEAD: Looking at Neighboring Rows



| month | revenue | prev_month |
|-------|---------|------------|
| Jan   | 1000    | —          |
| Feb   | 1200    | 1000       |
| Mar   | 900     | 1200       |
| Apr   | 1500    | 900        |

- `LAG(revenue) OVER (ORDER BY month)` — fetches the PREVIOUS row's value into the current row.
- `LEAD(revenue) OVER (ORDER BY month)` — fetches the NEXT row's value instead.
- Both accept an optional second argument for how many rows back/forward, and a third for a default when there is no such row.
- Classic use: month-over-month change = `revenue - LAG(revenue) OVER (ORDER BY month)`.

# Running Totals With Frame Clauses



| month | revenue | running_total |
|-------|---------|---------------|
| Jan   | 1000    | 1000          |
| Feb   | 1200    | 2200          |
| Mar   | 900     | 3100          |
| Apr   | 1500    | 4600          |

- `SUM(revenue) OVER (ORDER BY month ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)`
- The frame clause defines exactly which rows, relative to the current one, get included in the calculation.
- `UNBOUNDED PRECEDING AND CURRENT ROW` = 'every row up to and including this one' — that's a running total.
- With `ORDER BY` but no explicit frame, this exact 'running total' frame is actually the default for aggregate window functions.

# String, Date, and Type Functions You'll Use Constantly

| Function                   | What it does                       |
|----------------------------|------------------------------------|
| LOWER(x) / UPPER(x)        | Change case                        |
| TRIM(x)                    | Remove leading/trailing whitespace |
| SUBSTRING(x FROM a FOR b)  | Extract part of a string           |
| LENGTH(x)                  | Character count                    |
| NOW() / CURRENT_DATE       | Current timestamp / date           |
| AGE(date)                  | Interval between a date and now    |
| EXTRACT(YEAR FROM date)    | Pull out one part of a date        |
| x::type or CAST(x AS type) | Convert a value's type explicitly  |

## CASE: If/Else Inside a Query

```
SELECT name, price_cents,  
       CASE  
         WHEN price_cents < 2000 THEN 'budget'  
         WHEN price_cents < 10000 THEN 'mid-range'  
         ELSE 'premium'  
       END AS price_tier  
FROM products;
```

*CASE evaluates each WHEN condition top to bottom and returns the result tied to the first match; ELSE is the fallback if nothing matches (defaults to NULL if omitted).*

# Transactions and ACID

- A transaction groups multiple statements into one all-or-nothing unit: BEGIN; ...statements...; COMMIT; (or ROLLBACK; to undo).
- Atomicity — every statement in the transaction succeeds, or none of them take effect at all.
- Consistency — the database moves from one valid state to another; constraints are never violated at commit.
- Isolation — concurrent transactions don't see each other's uncommitted changes.
- Durability — once committed, changes survive even a crash immediately after.

## EXAMPLE

```
BEGIN;  
UPDATE accounts SET balance = balance  
- 100  
  WHERE id = 1;  
UPDATE accounts SET balance = balance  
+ 100  
  WHERE id = 2;  
COMMIT;
```

# Indexes: Why Queries Get Fast

- Without an index, PostgreSQL must scan every single row to find matches — a sequential scan, which gets slow as tables grow.
- An index is a separate, sorted data structure that lets the database jump straight to matching rows, much like a book's index lets you skip to a page instead of reading cover to cover.
- `CREATE INDEX idx_orders_customer ON orders(customer_id);` speeds up any query filtering or joining on `customer_id`.
- A primary key automatically gets an index for free — that's part of how uniqueness is enforced efficiently.
- Indexes aren't free: they speed up reads but slightly slow down writes (`INSERT/UPDATE/DELETE`), since the index itself must be kept up to date.

## Sequential Scan

PostgreSQL reading every row of a table in order, checking each against the query's condition. Fine for small tables; the thing indexes exist to avoid on large ones.

# Intermediate SQL: Quick Reference

| Tool              | Use it when...                                                                    |
|-------------------|-----------------------------------------------------------------------------------|
| Subquery          | You need a value/set computed from another query, used once                       |
| CTE (WITH)        | A subquery is reused, or the query is easier to read broken into named steps      |
| Recursive CTE     | Walking a hierarchy or graph of unknown depth                                     |
| Window function   | You need per-row context (rank, running total, prev/next) without collapsing rows |
| View              | A complex query needs a simple, reusable name                                     |
| Materialized View | That same query is expensive and can tolerate slightly stale data                 |

MODULE 6

# 06

## Advanced PostgreSQL

The features that set PostgreSQL apart: JSONB, full-text search, procedures, triggers, and tuning.

- Index types beyond B-tree, and EXPLAIN ANALYZE
- JSONB, arrays, and full-text search
- PL/pgSQL functions and triggers
- Security, schemas, partitioning, and backups

# Index Types: The Right Tool for the Job

PostgreSQL supports several index structures, each optimized for different kinds of queries.

| Index type       | Best for                                                          |
|------------------|-------------------------------------------------------------------|
| B-tree (default) | Equality and range queries: =, <, >, BETWEEN, ORDER BY            |
| Hash             | Simple equality only (=); rarely chosen over B-tree today         |
| GIN              | Full-text search, JSONB containment, arrays (many values per row) |
| GiST             | Geometric data, ranges, nearest-neighbor searches                 |
| BRIN             | Very large tables with naturally sorted data (e.g., timestamps)   |

# EXPLAIN and EXPLAIN ANALYZE

```
EXPLAIN SELECT * FROM orders WHERE customer_id = 42;
-- Shows the PLAN Postgres intends to use, without running the query

EXPLAIN ANALYZE SELECT * FROM orders WHERE customer_id = 42;
-- Actually RUNS the query, and shows real timing alongside the plan

--
--                                QUERY PLAN
--
-- Seq Scan on orders  (cost=0.00..18.50 rows=5 width=44)
--
--                        (actual time=0.02..0.15 rows=4 loops=1)
--
--   Filter: (customer_id = 42)
```

# Performance Tuning Basics

- Index the columns you actually filter, join, and sort on — not every column; unused indexes only slow down writes for no benefit.
- Avoid `SELECT *` in application code — fetching unused columns wastes I/O and network bandwidth.
- Watch for functions wrapped around indexed columns in `WHERE` (e.g. `WHERE LOWER(email) = ...`) — this can prevent the index from being used unless a matching expression index exists.
- N+1 query patterns (looping in application code, issuing one query per row) are a common performance killer — prefer a single `JOIN` or a batched query instead.
- Always measure with `EXPLAIN ANALYZE` before and after a change — intuition about performance is frequently wrong.

## QUICK WINS

- Add missing indexes on FK/`WHERE` columns
- Avoid `SELECT *`
- Batch instead of looping queries
- Measure, don't guess

# JSON and JSONB

```
CREATE TABLE events (  
    id          SERIAL PRIMARY KEY,  
    payload JSONB NOT NULL  
);  
  
INSERT INTO events (payload)  
VALUES ('{"type": "click", "user": {"id": 42, "vip": true}}');  
  
SELECT payload->'user'->>'id' AS user_id  
FROM events  
WHERE payload->'user'->>'vip' = 'true';
```

*JSONB stores JSON in a parsed, indexable binary format (preferred over plain JSON almost always). -> returns a JSON value; ->> returns it as text. A GIN index makes containment queries (@>) fast even on huge tables.*

# Array Columns

```
CREATE TABLE posts (  
    id      SERIAL PRIMARY KEY,  
    title TEXT,  
    tags   TEXT[]  
);  
  
INSERT INTO posts (title, tags)  
VALUES ('Intro to SQL', ARRAY['sql', 'beginner', 'database']);  
  
SELECT * FROM posts WHERE 'sql' = ANY(tags);  
SELECT * FROM posts WHERE tags @> ARRAY['sql','beginner'];  -- contains both
```

PostgreSQL allows any column type to be an array of that type. Handy for small, unordered lists like tags — though a proper junction table (Module 2) is usually still better once the ~~table~~ table has its own attributes or must be queried heavily.

# Built-in Full Text Search

```
SELECT title FROM articles
WHERE to_tsvector('english', body) @@ to_tsquery('english', 'database & performance');

-- Indexed for speed on large tables:
CREATE INDEX idx_articles_fts
  ON articles USING GIN (to_tsvector('english', body));
```

*to\_tsvector* breaks text into normalized, searchable lexemes (handling stemming: 'running' matches 'run'); *to\_tsquery* parses a search expression. @@ tests whether they match. **Full-Text Search** is a built-in search engine, built into Postgres, no external service required.

# Stored Functions

```
CREATE FUNCTION total_spent(p_customer_id INT)
RETURNS INTEGER AS $$
DECLARE
    v_total INTEGER;
BEGIN
    SELECT COALESCE(SUM(total_cents), 0) INTO v_total
    FROM orders
    WHERE customer_id = p_customer_id;

    RETURN v_total;
END;
$$ LANGUAGE plpgsql;

SELECT total_spent(42);
```

# Triggers: Code That Runs Automatically

```
CREATE FUNCTION set_updated_at() RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = now();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_orders_updated
BEFORE UPDATE ON orders
FOR EACH ROW
EXECUTE FUNCTION set_updated_at();
```

*A trigger fires a function automatically BEFORE or AFTER an INSERT/UPDATE/DELETE. This example auto-stamps updated\_at on every change — no application code has to manually set the timestamp.*

# Users, Roles, and Privileges

## Roles

- PostgreSQL has no separate 'users' concept — everything is a ROLE, which can log in (like a user) or not (like a group).
- `CREATE ROLE app_user WITH LOGIN PASSWORD '...';`
- Roles can be members of other roles, inheriting their privileges.

## Privileges

- `GRANT SELECT, INSERT ON orders TO app_user;`
- `REVOKE` removes a previously granted privilege.
- Principle of least privilege: give each role exactly the access it needs — never grant superuser to an application's connection.

# Schemas: Namespacing Inside a Database

- A schema is a namespace within a database that groups related tables together — not to be confused with 'schema' meaning overall table design.
- Every database starts with a default schema called public.
- `CREATE SCHEMA billing;` then `CREATE TABLE billing.invoices (...);` keeps billing tables separate from, say, analytics tables.
- Fully qualified names look like `schema.table`, e.g. `billing.invoices` — the `search_path` setting controls which schemas are checked when you write just `table` (no prefix).
- Useful for multi-tenant systems, separating concerns in a large application, or isolating a reporting/analytics area from transactional tables.

## **search\_path**

An ordered list of schemas Postgres checks, in order, to resolve an unqualified table name. Defaults to "\$user", public.

# Table Partitioning

- Partitioning splits one logical table into several physical pieces behind the scenes, while queries still address it as a single table.
- Range partitioning is the most common pattern: e.g., split a huge 'events' table by month, so each partition only holds one month's rows.
- Queries that filter by the partition key (e.g. WHERE event\_date >= '2026-01-01') only need to scan the relevant partition(s) — dramatically faster on huge tables.
- Also simplifies maintenance: dropping an old month of data becomes DROP the old partition, instantly, instead of a slow DELETE.
- A design decision for tables reaching tens of millions of rows or more — not needed for small-to-medium applications.

## SYNTAX SKETCH

```
CREATE TABLE events (  
    ...  
) PARTITION BY RANGE (event_date);  
  
CREATE TABLE events_2026_01  
    PARTITION OF events  
    FOR VALUES  
    FROM ('2026-01-01') TO ('2026-02-01');
```

# Extensions and Foreign Data Wrappers

- PostgreSQL is deliberately extensible — `CREATE EXTENSION` adds optional functionality without changing the core database.
- `postgis` — adds full geographic/spatial data types and queries.
- `pg_stat_statements` — tracks execution statistics for every query run, invaluable for performance tuning.
- `uuid-oss` / `pgcrypto` — UUID generation and cryptographic functions.
- Foreign Data Wrappers (FDWs) let PostgreSQL query EXTERNAL data sources — another Postgres server, MySQL, even a CSV file — as if they were local tables.

## SYNTAX

```
CREATE EXTENSION IF NOT EXISTS  
postgis;  
CREATE EXTENSION IF NOT EXISTS  
pgcrypto;
```

# Backing Up and Restoring a Database

1

## **`pg_dump mydb > backup.sql`**

Exports the entire database as a plain-text SQL script (or a compressed custom format with `-Fc`).

2

## **Store it safely**

Off the same server — cloud storage, another machine, a dedicated backup service.

3

## **`createdb restored_db`**

Create an empty destination database to restore into.

4

## **`psql restored_db < backup.sql`**

Replays every statement, recreating the schema and all data (`pg_restore` for the custom `-Fc` format).

5

## **Test it periodically**

A backup you've never restored from is not a verified backup — practice the full restore process before you need it in an emergency.

# Replication Basics

- Replication keeps a live, continuously-updated COPY of a database on one or more additional servers.
- Streaming replication is PostgreSQL's built-in method: a primary server ships its write-ahead log (WAL) to one or more standby (replica) servers in near real time.
- Read replicas let you route read-only queries (reports, dashboards — Module 7!) to a standby, reducing load on the primary that handles writes.
- If the primary fails, a standby can be promoted to take over — the foundation of high availability.
- A deep operational topic in its own right; the goal here is recognizing the term and its purpose, not configuring it.

## WAL

Write-Ahead Log — PostgreSQL records every change here BEFORE applying it to the actual data files. This guarantees durability (the 'D' in ACID) and is also what replication streams to standby servers.

# Database Design: Do's and Don'ts

## Do

- Normalize first, denormalize deliberately, later, if measured performance demands it.
- Name tables/columns consistently (snake\_case is the Postgres convention).
- Add NOT NULL and CHECK constraints wherever a rule is knowable up front.
- Index foreign key columns — Postgres does NOT do this automatically.

## Don't

- Store comma-separated lists in a single column (violates 1NF).
- Use SELECT \* in application code.
- Trust application code alone to enforce data rules — the database should too.
- Add indexes 'just in case' without a query that needs them.

# Advanced PostgreSQL: Quick Reference

| Feature              | One-line purpose                                        |
|----------------------|---------------------------------------------------------|
| GIN index            | Fast search inside JSONB, arrays, and full-text vectors |
| EXPLAIN ANALYZE      | See and measure exactly how a query executes            |
| JSONB                | Flexible, indexable semi-structured data inside SQL     |
| PL/pgSQL function    | Reusable server-side logic                              |
| Trigger              | Code that runs automatically on INSERT/UPDATE/DELETE    |
| Schema               | Namespace for grouping related tables                   |
| Partitioning         | Split a huge table for speed and easier maintenance     |
| pg_dump / pg_restore | Backup and recovery                                     |

# From Database to Dashboard

- You can now design, build, query, and tune a real PostgreSQL database from scratch.
- The final technical module connects everything you've learned to a live, browser-based dashboard — the payoff of building a well-structured database.
- We'll build a small Node.js/Express API that runs SQL queries against Postgres, and a plain HTML/JavaScript front end that renders the results as KPI cards, a chart, and a table.

## 7

Dashboards Module Starts Next

Node.js · Express · fetch() · Chart.js · live queries

MODULE 7

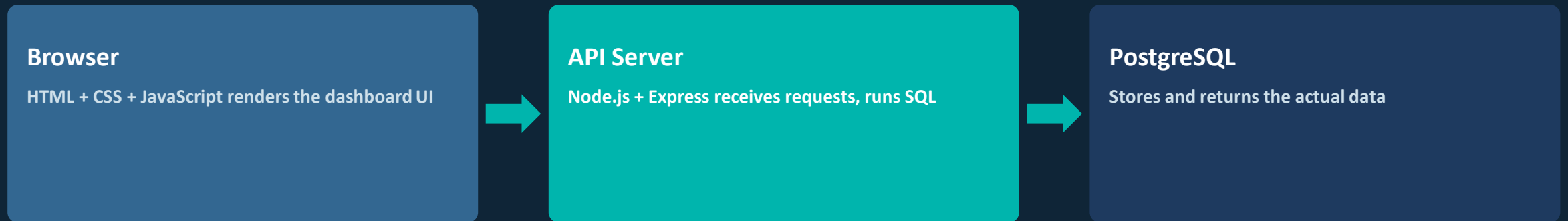
# 07

## Building Dashboards with JavaScript & HTML

Serving live PostgreSQL data to a browser architecture, API, and front end, end to end.

- The browser-to-database architecture
- A Node.js + Express API querying Postgres
- Fetching and rendering data in plain HTML/JS
- Charts, filters, and safe, parameterized queries

# How a Browser Talks to PostgreSQL



**Browser sends an HTTP request → API queries Postgres → API sends JSON back → browser renders it**

- A browser can NEVER connect to PostgreSQL directly — for security and protocol reasons, there must always be a server layer in between.
- That middle layer (commonly called “the backend” or “the API”) is the only thing that holds real database credentials.
- This exact three-layer shape — client, API, database — is the foundation of the overwhelming majority of web applications, not just dashboards.

# Connecting Node.js to PostgreSQL

```
$ npm init -y
$ npm install pg express

// db.js
const { Pool } = require('pg');

const pool = new Pool({
  host: process.env.PGHOST,
  user: process.env.PGUSER,
  password: process.env.PGPASSWORD,
  database: process.env.PGDATABASE,
  port: 5432
});

module.exports = pool;
```

*pg (node-postgres) is the standard PostgreSQL driver for Node.js. A Pool manages a set of reusable database connections — much more efficient than opening a brand-new connection before every request.*

# A Minimal Express Server

```
// server.js
const express = require('express');
const pool = require('./db');

const app = express();
app.use(express.static('public')); // serves our HTML/JS/CSS files

app.get('/api/health', (req, res) => {
  res.json({ status: 'ok' });
});

app.listen(3000, () => console.log('Server running on port 3000'));
```

# An Endpoint That Queries Postgres

```
app.get('/api/top-customers', async (req, res) => {
  try {
    const result = await pool.query(`
      SELECT c.first_name, SUM(o.total_cents) AS total
      FROM customers c
      JOIN orders o ON o.customer_id = c.id
      GROUP BY c.id
      ORDER BY total DESC
      LIMIT 5
    `);
    res.json(result.rows);
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: 'Query failed' });
  }
})
```

*`pool.query()` runs SQL and returns rows as plain JavaScript objects on `result.rows` — `res.json()` sends them to the browser as JSON automatically.*

# Environment Variables and Connection Security

- Database credentials (host, username, password) must NEVER be hard-coded directly in source files, especially ones committed to version control.
- Environment variables (accessed in Node as `process.env.NAME`) keep secrets outside the codebase — typically loaded from a local, git-ignored `.env` file via the `dotenv` package.
- In production, these values are set through the hosting platform's configuration, not files at all.
- The API server's database role (Module 6, slide 6.9) should follow least privilege — a dashboard almost never needs anything beyond `SELECT`.

## **.ENV (NEVER COMMITTED)**

```
PGHOST=localhost  
PGUSER=dashboard_reader  
PGPASSWORD=*****  
PGDATABASE=course_db
```

# The Dashboard's HTML Skeleton

```
<!DOCTYPE html>
<html>
<head>
  <title>Sales Dashboard</title>
  <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
</head>
<body>
  <h1>Sales Dashboard</h1>

  <div id="kpi-cards"></div>
  <canvas id="revenue-chart"></canvas>
  <table id="customers-table"></table>

  <script src="dashboard.js"></script>
</body>
</html>
```

*Empty containers (div, canvas, table) act as placeholders — JavaScript will fill them with real data after the page loads.*

# The fetch() API

```
async function loadTopCustomers() {  
  const response = await fetch('/api/top-customers');  
  const customers = await response.json();  
  return customers;  
}  
  
loadTopCustomers().then(customers => {  
  console.log(customers);  
  // [{ first_name: 'Ada', total: 84900 }, ...]  
});
```

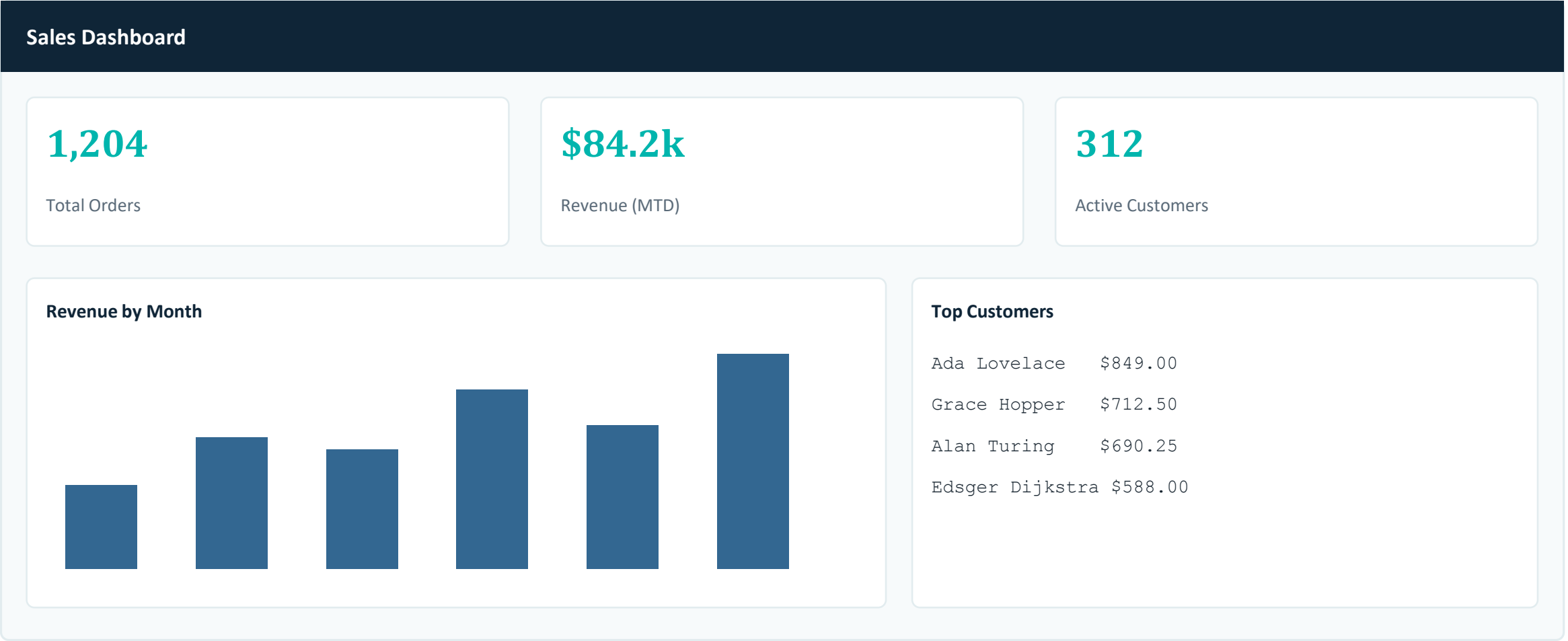
# Turning JSON into an HTML Table

```
function renderTable(rows) {  
  const table = document.getElementById('customers-table');  
  table.innerHTML = '<tr><th>Customer</th><th>Total</th></tr>' +  
    rows.map(r => `  
      <tr>  
        <td>${r.first_name}</td>  
        <td>${(r.total / 100).toFixed(2)}</td>  
      </tr>  
    `).join('');  
}  
  
loadTopCustomers().then(renderTable);
```

# Adding a Chart with Chart.js

```
async function renderRevenueChart() {  
  const res = await fetch('/api/monthly-revenue');    // uses window functions from Module  
5!  
  const data = await res.json();  
  
  new Chart(document.getElementById('revenue-chart'), {  
    type: 'line',  
    data: {  
      labels: data.map(d => d.month),  
      datasets: [{ label: 'Revenue', data: data.map(d => d.total) }]  
    }  
  });  
}
```

# The Full Dashboard, Assembled



# User Filters and SQL Injection

```
// DANGEROUS — never do this:
app.get('/api/customer', (req, res) => {
  pool.query(`SELECT * FROM customers WHERE email = '${req.query.email}'`);
  // an attacker could send: ?email=' OR '1'='1
});

// SAFE — use parameterized queries:
app.get('/api/customer', async (req, res) => {
  const result = await pool.query(
    'SELECT * FROM customers WHERE email = $1',
    [req.query.email]
  );
  res.json(result.rows);
});
```

SQL injection happens when raw user input is pasted directly into a query string. `$1`, `$2...` placeholders (parameterized queries) tell Postgres to treat the value strictly as DATA, never as executable SQL. **Always use them for any user-supplied filter.**

# Polling vs. WebSockets

## Polling

Browser calls `fetch()` every N seconds on a timer (`setInterval`). Simple, works everywhere, slight delay.



## WebSockets

A persistent, two-way connection; the server pushes updates the instant new data exists. More complex, near-instant.

- For most internal dashboards, polling every 30-60 seconds with `setInterval(loadDashboard, 30000)` is simple, reliable, and entirely sufficient.
- Reach for WebSockets (or Postgres's own LISTEN/NOTIFY, paired with a WebSocket server) only when updates genuinely need to appear within a second or two — live chat, trading tickers, collaborative editing.
- Start simple: polling is the right default choice unless you have a proven, specific need for true real-time updates.

# Deploying Your Dashboard

- The database, the API, and the static front-end files can be deployed separately or together, depending on the hosting platform.
- Managed Postgres hosting (e.g. a cloud provider's database service) handles backups, updates, and scaling for you — a common choice over self-managing a server.
- The Node/Express API is typically deployed as a small web service process, configured with the same environment variables discussed in slide 7.5.
- Static HTML/JS/CSS files can be served directly by Express (as shown in 7.3) or hosted separately on a static file host — either works.
- Always use HTTPS in production, and double-check that database credentials and ports are never exposed to the public internet directly — only your API server should be able to reach Postgres.

## PRE-LAUNCH CHECKLIST

- Secrets in env vars, not code
- Read-only DB role for the dashboard
- HTTPS enabled
- Database not publicly reachable
- Backups tested (Module 6)

MODULE 8

# 08

## Wrap-Up & Reference

A map of everything you've covered, plus cheat sheets to keep close while you practice.

- The full course, at a glance
- SQL syntax quick reference
- Where to go from here

# From Set Theory to a Live Dashboard

| Module                 | The one idea to remember                                                        |
|------------------------|---------------------------------------------------------------------------------|
| 1. Set Theory          | A table is a set of rows; $\cup \cap - \times$ underlie UNION, JOIN, and EXCEPT |
| 2. Relational Theory   | Model with ER diagrams first; normalize to 3NF to kill redundancy               |
| 3. Getting Started     | CREATE TABLE + constraints turn theory into an enforced schema                  |
| 4. Core SQL            | SELECT / WHERE / JOIN / GROUP BY answer nearly any everyday question            |
| 5. Intermediate SQL    | CTEs and window functions solve what plain joins can't, cleanly                 |
| 6. Advanced PostgreSQL | JSONB, indexing, functions, and tuning make Postgres production-ready           |
| 7. Dashboards          | Browser → API → Postgres, with parameterized queries, ships real products       |

# SQL Syntax Quick Reference

| Clause                  | Purpose                                              |
|-------------------------|------------------------------------------------------|
| SELECT cols FROM table  | Choose columns from a table (relation)               |
| WHERE condition         | Filter individual rows                               |
| JOIN ... ON ...         | Combine rows from two tables by a matching condition |
| GROUP BY cols           | Bucket rows for aggregation                          |
| HAVING condition        | Filter groups, after aggregation                     |
| ORDER BY col [ASC DESC] | Sort the final result                                |
| LIMIT n OFFSET m        | Return only a page of results                        |
| WITH name AS (...)      | Define a reusable, named subquery (CTE)              |
| func(...) OVER (...)    | Per-row calculation across a window of rows          |

# Where To Go From Here

- Practice on a real dataset that interests you — motivation sticks far better than generic exercises. Import a CSV you care about and start asking it questions in SQL.
- Rebuild the Module 7 dashboard from scratch, unaided, using a different dataset — the single best test of whether the concepts have truly landed.
- Read PostgreSQL's official documentation ([postgresql.org/docs](https://postgresql.org/docs)) — it is unusually well-written and often the fastest path to a precise answer.
- Explore adjacent tools as you grow: an ORM (like Prisma or Sequelize) for larger applications, a migration tool for managing schema changes over time, and connection pooling (PgBouncer) for high-traffic production systems.
- Most of all: keep writing real queries against real data. Fluency in SQL, like any language, comes from use — not just study.



Course Complete

120 slides · 8 modules · set theory to a live dashboard